

MODELO RELACIONAL Y PROCESO DE NORMALIZACIÓN

Sistema de Gestión de Reparaciones de PC "TecnoService"

Módulo Programador | Hito 2 - Evidencia 5 | Equipo Bit&Fix (Comisión B1)

1. Del modelo E-R al modelo relacional

El diagrama entidad-relación (Evidencia 4, corregido) define 5 entidades y 4 relaciones. Al pasar al modelo relacional, cada entidad se convierte en una tabla, y las relaciones se resuelven según su cardinalidad:

- CLIENTE (1) -- posee -- (N) EQUIPO → relación 1:N: la clave primaria de CLIENTE (id_cliente) se agrega como clave foránea en EQUIPO.
- EQUIPO (1) -- registra -- (N) REPARACION → relación 1:N: id_equipo se agrega como clave foránea en REPARACION.
- TECNICO (1) -- atiende -- (N) REPARACION → relación 1:N: id_tecnico se agrega como clave foránea en REPARACION.
- REPARACION (N) -- utiliza -- (M) REPUESTO → relación N:M: no se puede resolver con una sola clave foránea. Se crea una tabla intermedia, DETALLE_REPARACION, cuya clave primaria es compuesta (id_reparacion, id_repuesto) y que además guarda los atributos propios de la relación: cantidad y precio_unitario_aplicado.

2. Modelo relacional resultante

Notación: subrayado = clave primaria (PK); cursiva = clave foránea (FK).

CLIENTE (id_cliente, dni, nombre, telefono)

EQUIPO (id_equipo, tipo, marca, modelo, numero_serie, *id_cliente**)

TECNICO (id_tecnico, dni, nombre, apellido, especialidad, estado)

REPARACION (id_reparacion, fecha_ingreso, fecha_salida, falla_reportada, diagnostico, costo_estimado, costo_total, estado, *id_equipo**, *id_tecnico**, presupuesto_aprobado)

REPUESTO (id_repuesto, descripcion, precio_unitario, stock, stock_minimo)

DETALLE_REPARACION (*id_reparacion**, *id_repuesto**, cantidad, precio_unitario_aplicado)

(* = clave foránea)

3. Proceso de normalización

Se partió directamente de un modelo ya pensado en tablas separadas por entidad, pero igual se verificó cada tabla contra las tres formas normales, para justificar por qué no hace falta descomponerlas más.

3.1 Primera Forma Normal (1FN)

Una tabla está en 1FN si todos sus atributos son atómicos (no repetidos ni multivaluados) y cada fila es identificable por una clave primaria.

Se verificó, en particular, la tabla REPARACION: en un diseño inicial e incorrecto, podría haberse guardado "los repuestos usados" como una lista dentro de una sola celda de texto (ej: "SSD 480GB, Pasta Térmica"). Eso violaría la 1FN por ser un atributo multivaluado. La solución fue extraer esa información a la tabla independiente

DETALLE_REPARACION, con una fila por cada repuesto usado en cada reparación. El resto de las tablas ya nacieron con atributos atómicos (ningún campo contiene listas ni valores compuestos).

3.2 Segunda Forma Normal (2FN)

Una tabla está en 2FN si está en 1FN y, además, en las tablas con clave primaria compuesta, todo atributo no clave depende de la clave completa (no de una parte de ella).

La única tabla con clave primaria compuesta es DETALLE_REPARACION (id_reparacion, id_repuesto). Se analizaron sus dos atributos no clave:

- cantidad: depende de la combinación específica de una reparación y un repuesto (cuántas unidades de ESE repuesto se usaron en ESA reparación). Depende de la clave completa. Cumple 2FN.
- precio_unitario_aplicado: depende del momento en que se usó ese repuesto en esa reparación puntual (el precio de catálogo puede cambiar con el tiempo, y este campo "congela" el precio vigente en ese momento). No depende solo de id_repuesto —si dependiera solo de eso, sería una redundancia del precio_unitario de REPUESTO—, sino de la combinación reparación + repuesto. Depende de la clave completa. Cumple 2FN.

3.3 Tercera Forma Normal (3FN)

Una tabla está en 3FN si está en 2FN y, además, ningún atributo no clave depende de otro atributo no clave (sin dependencias transitivas).

Se revisó cada tabla buscando atributos que en realidad dependieran de otro atributo no clave en lugar de la clave primaria:

- EQUIPO: en un diseño anterior (Evidencia 4, versión inicial) esta tabla incluía falla y estado. Se detectó que esos dos atributos en realidad dependen de CADA VISITA al taller, no del equipo en sí (dependían transitivamente del hecho de "qué reparación puntual se esté describiendo", algo que ni siquiera estaba en la tabla). Se migraron a REPARACION, donde sí dependen directamente de id_reparacion. Este fue el ajuste de normalización más importante del proyecto.
- REPARACION: costo_total podría pensarse como derivado de sumar costo_estimado más el total de detalle_reparacion, pero se decidió mantenerlo como atributo propio (no calculado) porque representa el monto final ya facturado, que puede incluir ajustes manuales y no debe recalcularse solo a partir de otras tablas. No genera una dependencia transitiva porque no depende de otro atributo no clave de la misma tabla, sino que es un dato ingresado por el técnico/administrador al cerrar la orden.
- El resto de las tablas (CLIENTE, TECNICO, REPUESTO) no presentan atributos que dependan de otro atributo no clave: todos dependen directamente de su respectiva clave primaria.

4. Conclusión

El modelo relacional definitivo, reflejado en el diccionario de datos y en script_tecnoservice.sql, cumple 3FN. El ajuste más relevante del proceso de normalización fue trasladar falla y estado desde EQUIPO hacia REPARACION, justamente porque en el diseño original generaban una dependencia incorrecta (esos datos describen una reparación puntual, no el equipo como activo permanente).